

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 5

Выполнил:

Берулава Леон Алхасович

К3439

Проверил:

Добряков Д. И.

Санкт-Петербург

2025 г.

Задача

- подключить и настроить rabbitMQ/kafka;
- реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Ход работы

В рамках работы был выбран RabbitMQ в качестве message broker для реализации асинхронного межсервисного взаимодействия в микросервисной архитектуре кулинарного приложения. RabbitMQ развернут в отдельном Docker контейнере с использованием официального образа `rabbitmq:3-management` который включает веб-интерфейс управления для мониторинга очередей, `exchanges` и сообщений в реальном времени.

Конфигурация RabbitMQ в `docker-compose.yml` включает настройку пользователя и пароля через `environment` переменные `RABBITMQ_DEFAULT_USER` и `RABBITMQ_DEFAULT_PASS` для обеспечения безопасного доступа к брокеру сообщений. Контейнер экспонирует порт 5672 для AMQP протокола используемого приложениями для отправки и получения сообщений а также порт 15672 для веб-интерфейса управления доступного по адресу <http://localhost:15672>.

Настройка `health check` для RabbitMQ контейнера обеспечивает проверку готовности сервиса к обработке сообщений через команду `rabbitmqctl status` что позволяет другим сервисам ожидать полной инициализации брокера перед началом работы. Persistent volume `rabbitmq_data` используется для сохранения конфигурации и сообщений между перезапусками контейнера что обеспечивает надежность доставки сообщений.

Сетевая конфигурация включает RabbitMQ в общую Docker сеть `kitchen-network` что позволяет всем микросервисам подключаться к брокеру по имени контейнера `rabbitmq` без необходимости использования IP адресов. Каждый микросервис получает параметры подключения через `environment` переменные `RABBITMQ_HOST` и `RABBITMQ_PORT` что обеспечивает гибкость конфигурации для различных окружений развертывания.

Архитектура межсервисного взаимодействия построена на принципах Event-Driven Architecture с использованием паттерна `publish-subscribe` для обеспечения `loose coupling` между микросервисами. Реализован

универсальный `MessageService` класс который инкапсулирует логику подключения к `RabbitMQ`, создания `exchanges`, очередей и обработки сообщений с автоматической обработкой ошибок и переподключением при сбоях сети.

`MessageService` использует `fanout exchange` с именем `user_events` для broadcast доставки событий всем заинтересованным сервисам что позволяет легко добавлять новых подписчиков без изменения публикующих сервисов. Каждый сервис создает собственную `durable` очередь с уникальным именем включающим название сервиса что обеспечивает изоляцию сообщений и возможность независимой обработки событий разными сервисами с различной скоростью.

Публикация событий реализована через метод `publishUserCreated` который принимает данные пользователя и создает стандартизированное сообщение с типом события, временной меткой, идентификатором источника и полезной нагрузкой. Сообщения помечаются как `persistent` что гарантирует их сохранение на диске и доставку даже при перезапуске `RabbitMQ` сервера обеспечивая надежность системы.

Подписка на события реализована через метод `subscribeToUserCreated` который создает `consumer` для обработки входящих сообщений с автоматическим `acknowledgment` при успешной обработке и `negative acknowledgment` с отклонением сообщения при ошибках. Фильтрация сообщений выполняется на уровне `consumer` по типу события и источнику что предотвращает обработку сервисом собственных событий и зацикливание.

`Graceful shutdown` реализован через перехват системных сигналов `SIGTERM` и `SIGINT` с корректным закрытием соединений и каналов `RabbitMQ` что предотвращает потерю сообщений при остановке сервисов. `Error handling` включает `retry` логику для временных сетевых сбоев и подробное логирование ошибок для упрощения диагностики проблем межсервисного взаимодействия.

Основной сценарий использования реализован для автоматического создания приветственного рецепта при регистрации нового пользователя что демонстрирует асинхронную обработку бизнес-событий без блокирования основного потока выполнения. `User Service` при успешном создании пользователя публикует событие `USER_CREATED` с данными

пользователя включая идентификатор, имя пользователя, email и личную информацию.

Recipe Service подписывается на события `USER_CREATED` и асинхронно создает персонализированный приветственный рецепт для нового пользователя включающий приветствие по имени и простой рецепт блинчиков как стартовый контент. Обработка события включает валидацию данных пользователя, создание структуры рецепта с ингредиентами и инструкциями в JSON формате и сохранение в базе данных с привязкой к пользователю.

Каталожный сервис также может подписываться на пользовательские события для ведения статистики регистраций, создания персональных рекомендаций категорий блюд или отправки welcome emails через интеграцию с внешними email сервисами. Расширяемость архитектуры позволяет легко добавлять новые типы событий такие как `RECIPE_CREATED`, `RECIPE_LIKED` или `USER_UPDATED` для реализации сложных бизнес-процессов.

Мониторинг очередей сообщений осуществляется через RabbitMQ Management UI где можно отслеживать количество сообщений в очередях, скорость обработки, ошибки доставки и производительность consumers. Метрики включают количество published, delivered и acknowledged сообщений что позволяет выявлять узкие места в обработке и оптимизировать производительность системы.

Вывод

Реализованная система очередей сообщений обеспечивает надежное асинхронное взаимодействие между микросервисами с гарантией доставки сообщений, возможностью горизонтального масштабирования обработчиков событий и простотой добавления новых сервисов в event-driven архитектуру без изменения существующего кода.